

DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Construyendo Agentes de Inteligencia Artificial

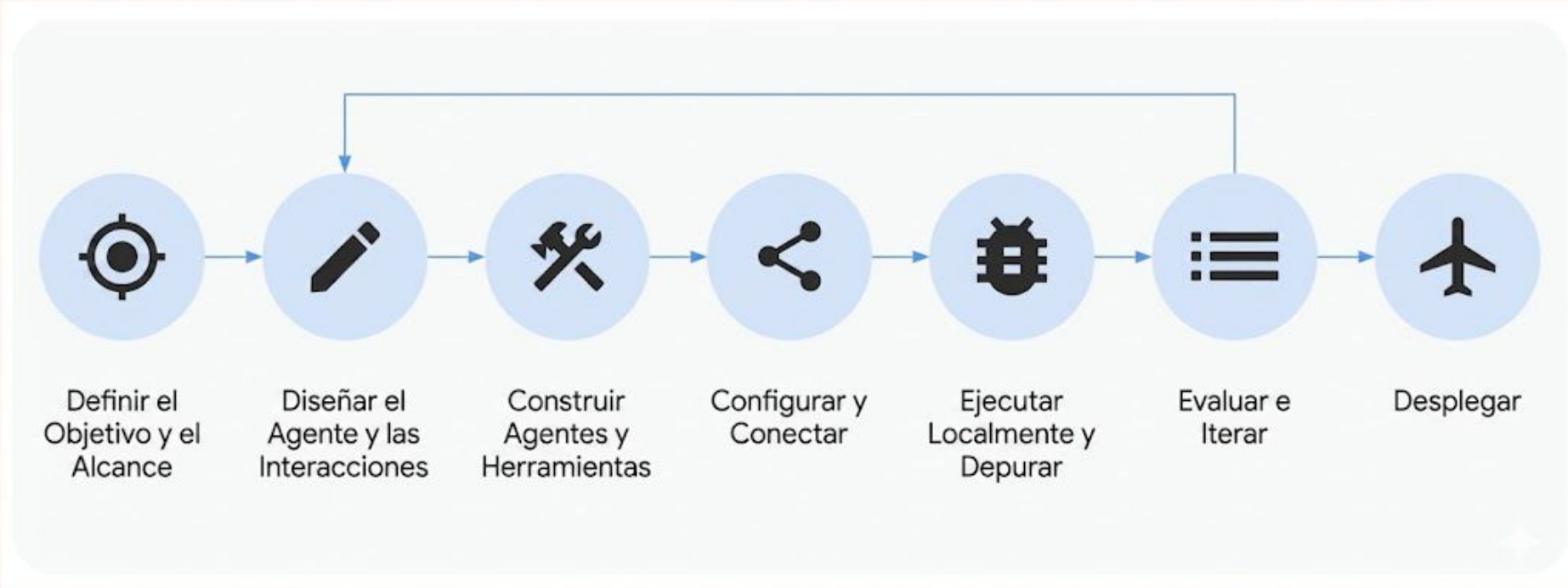
Día 2 : Diseñando Agentes

Martes 28 de Julio del 2026

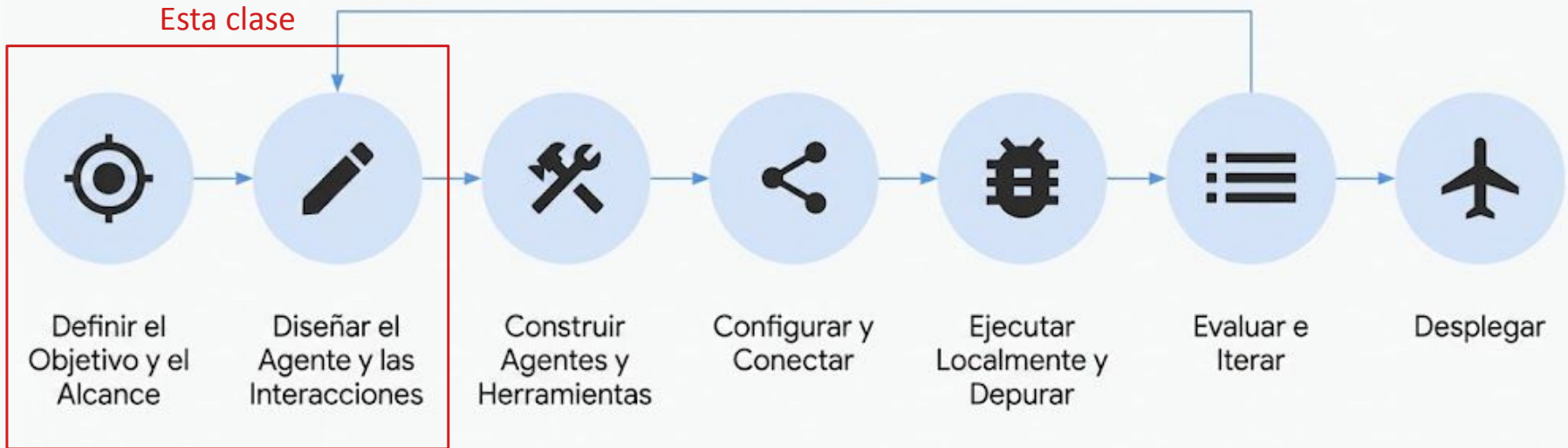
Repaso Clase 1



El Ciclo de Vida de Creación de Agentes



Proceso de Creación de Agentes





Definir el Objetivo y Alcance

Usos de los modelos de lenguaje

Negocios y Profesional

Soporte en la toma de decisiones, automatización de procesos documentales, tareas administrativas, chatbots profesionales y clasificación automática de tickets.

Marketing y Ventas

Automatización del feedback de clientes, predicción de comportamientos de mercado, asistencia en la revisión de CVs y procesos de reclutamiento.

Elaboración de Documentos

Uso de IA para la redacción y estructuración de informes, reportes corporativos, planes de entrenamiento e investigaciones detalladas de mercado.

Integración de Herramientas

Modelos de lenguaje actuando como asistentes integrados en suites de oficina, generación de código fuente y creación de imágenes o videos.

Medicina y Salud

Generación de resúmenes clínicos, análisis rápido de estudios complejos, procesamiento de historias clínicas y agentes de asistencia en triage sanitario.

Educación

Redefinición del aprendizaje mediante tutorías personalizadas, creación automatizada de contenidos, resúmenes interactivos y asistencia en corrección.

Desarrollo de Software

Generación y autocompletado de código, explicación de lógica compleja, documentación automática de proyectos y detección activa de errores.

Finanzas

Seguimiento en tiempo real de indicadores económicos, análisis avanzado de grandes volúmenes de datos y predicción de tendencias de mercado.

IA Conversacional y sus Categorías

Definición

IA Conversacional es un conjunto de tecnologías diseñadas para imitar o reemplazar las interacciones humanas usando el lenguaje escrito o hablado.

Ejemplos: chatbots, agentes virtuales, asistentes de IA y empleados digitales.

Evolución Tecnológica

Históricamente se usaban técnicas de NLU y flujos deterministas.

Actualmente se implementan **agentes híbridos** que combinan flujos deterministas con LLMs e IA Generativa.

01

Pregunta - Respuesta

Resuelven FAQs y responden a la pregunta directa del usuario, generalmente sin necesidad de seguimiento (follow-up).

02

Orientados a Procesos

Buscan lograr un objetivo transaccional (CRUD), como obtener saldos, agendar citas o derivar a procesos manuales.

03

Agentes de Ruteo

Su función principal es redirigir inteligentemente al usuario hacia otro agente especializado o un operador humano.



Un mismo agente conversacional puede combinar e integrar las **3 soluciones** de forma simultánea.

¿Cómo definir el objetivo y alcance?

Para lograr una definición precisa, debemos realizar estas actividades fundamentales:

- **Determinar usuarios:** Identificar quiénes interactuarán con el agente de manera directa.
- **Determinar los casos de uso:** Definir las situaciones concretas de negocio o tareas que resolverá el agente.
- **Relevar stakeholders:** Entrevistar personas que aporten detalles de usos y costumbres.
- **Investigar procesos:** Analizar los procesos actuales para hallar puntos de mejora (pain points) y definir como serán reemplazados por el agente.

Preguntas de Diagnóstico

01. ¿Cuáles son las dificultades de los usuarios?

Identifica los dolores actuales de las personas en su flujo.

02. ¿Qué se precisa que haga el agente?

Define claramente el rol y los límites del asistente conversacional.

03. ¿Qué información precisa el agente y qué esperan?

Establece los datos de entrada requeridos y las respuestas de salida. Se deberá pensar en la arquitectura integrada con las distintas funcionalidades técnicas que se puedan requerir. También, requerimientos blandos como el tono del agente y lineamientos de comunicación establecidos.

04. Impacto del cambio: ¿Cómo resuelven hoy el problema y cómo se comunicarán?

Determina la transición operativa y el canal adecuado del agente.

Requerimientos para la creación de Agentes

Al diseñar un agente, es clave ver en qué pasos del flujo conversacional se requiere:

01. Consultar herramientas

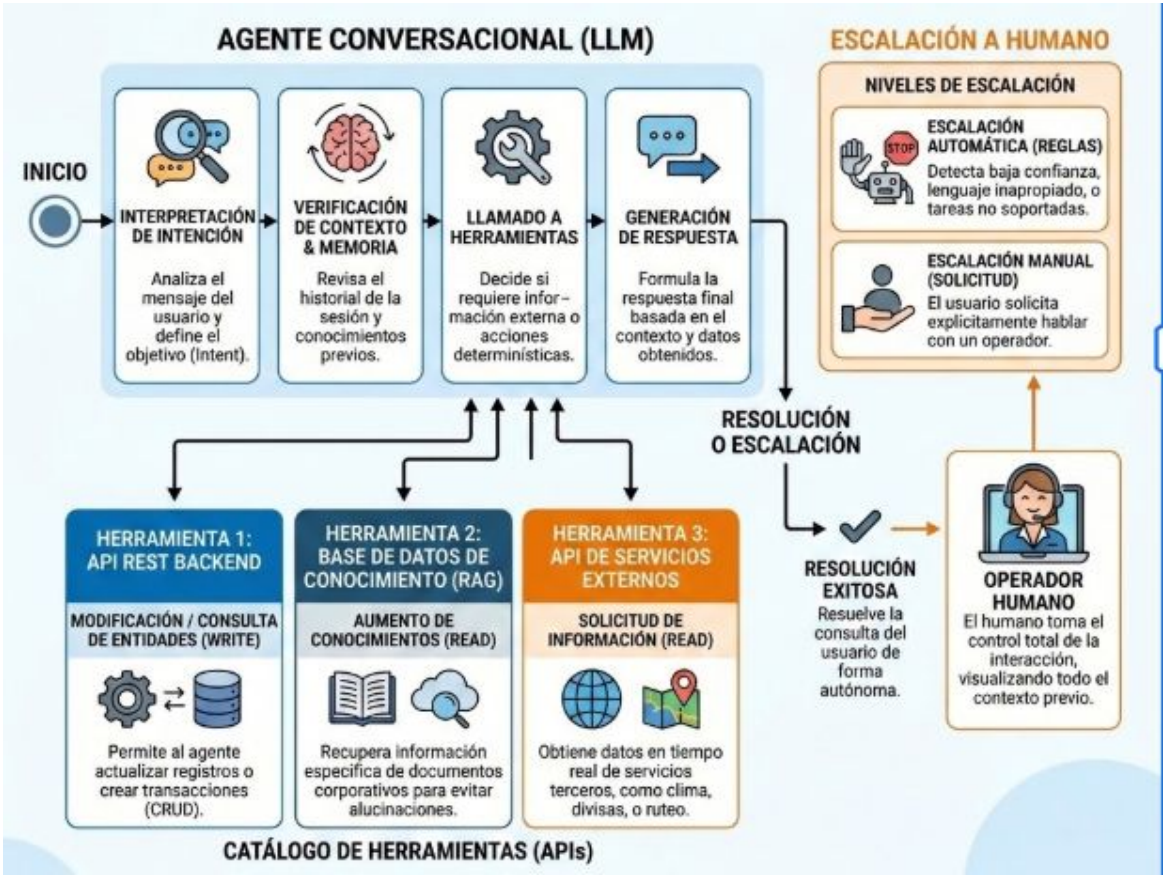
Llamar a APIs, bases de datos o servicios externos.

02. Usar información de contexto

Acceder al historial de sesión, memoria y conocimientos previos.

03. Escalar a un operador humano

Transferir la conversación ante casos complejos o no soportados.



Elegir el canal y forma de comunicación



1. TELÉFONO (VOZ)

- Llamadas directas
- Reconocimiento de voz
- Conversaciones naturales

**Ejemplo: "Hable con el agente..."*



2. WHATSAPP

- Mensajería rápida
- Respuestas automatizadas
- Integración multimedia

**Ejemplo: "Envíe un mensaje al..."*

3. PÁGINAS WEB (GUI)



- Widget de chat en vivo
- Navegación asistida
- Respuestas visuales

**Ejemplo: "Inicie el chat en..."*



4. TERMINAL (CLI)

```
user> hola
agent> ¡Hola! ¿En qué puedo ayudarte?
user> información sobre precios
agent> Claro, aquí tienes las tarifas
actuales...
```

- Comandos rápidos
- Interacción de texto puro
- Automatización de scripts

**Ejemplo: "Ejecute el agente..."*



Diseñar el Agente y las interacciones

Diseño Conversacional y Experiencia (UX)

Definiendo qué hace el agente

El diseño de un agente es el paso más importante de todo agente conversacional. Hay algunas definiciones genéricas que deberíamos extraer antemano antes de pensar los diálogos, la arquitectura del mismo o la implementación.

- Usuario: pensar quienes van a utilizar el agente
- Medio: a través de que tecnología van a poder conversar con el agente: voz o chat, whatsapp, página web, llamada telefónica, portal de la compañía, etc.
- Tareas y Flujos de trabajo (acciones): Tareas y por consiguiente, los flujos de trabajo que el agente va a atender. Por ejemplo, un flujo podría ser un asistente para responder preguntas de facturación, conocer saldos y pagar facturas.
- Tono, nombre, características: Definir la voz femenina, masculina, el tono, su misión y otras características por ejemplo, "usa un lenguaje cordial y simple para responder".

Diseñar los flujos e interacciones, conversación por conversación

Al definir los diferentes flujos y acciones que va a poder hacer el agente, podemos ir pensando como va a ser su interacción con los usuarios.

- Mensaje de bienvenida
- Si ofrece las acciones o indica cuales son las tareas que puede llevar a cabo
- Información adicional que puede ofrecer
- Determinación de las acciones que quiere llevar a cabo el usuario (intención)
- Determinar la información que precisa para realizar cada acción pedida por el usuario y cómo consultar solicitar / obtener los datos que precise.
- Comunicación de que la tarea fue completada
- Escalamientos a humanos en caso de no poder determinar la acción o que haya una tarea manual

Lo ideal es escribir varias conversaciones punta a punta, de los diferentes casos de uso, que vayan atravesando todos los flujos conversacionales.

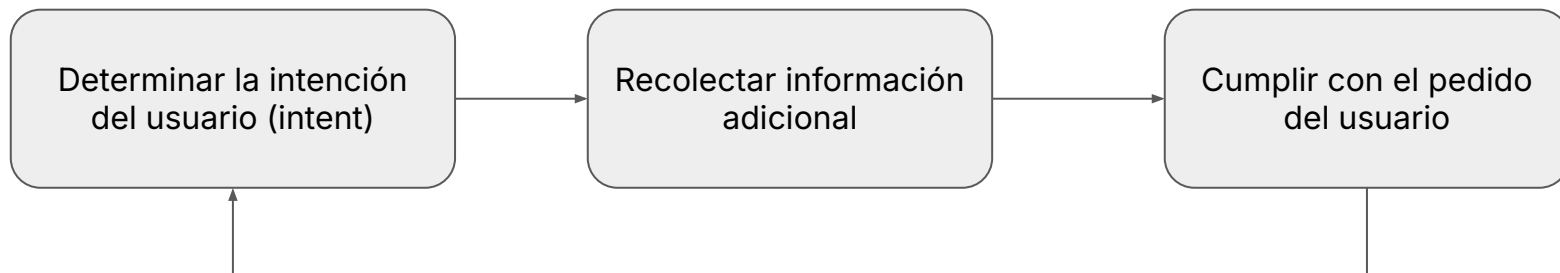
Dichas conversaciones deberán estar de alguna forma estandarizadas en tono, forma de solicitar información, etc. Los flujos deben compartir un mismo estilo para que de la sensación de ser un único agente.

Se sugiere analizar la cobertura, es decir asegurar que estos flujos capturen la totalidad de los casos de uso e incluso variaciones de formas de expresarse.

Diseñar el Agente: Root Agent, Flow, Steering e Intent

Hay cuatro términos fundamentales:

- Root Agent (Agente Raíz): Se trata del Agente que es raíz del árbol de Agentes, es donde inicia la conversación
- Steering: Proceso de redirigir el inicio del agente hacia algún flujo, muchas veces ocurre al inicio del agente y permite al usuario tomar el flujo correcto y las acciones que está buscando.
- Flow (Flujo): Camino, secuencia de pasos desde el inicio hasta el final de una conversación con el agente.
- Intent (Intención): Palabra o grupo de palabras que indican la "intención" del usuario en un punto dado de la conversación. Se refiere a interpretar y definir que quiere hacer el usuario, su intención o su objetivo (goal) con el agente. Por ejemplo, si el usuario dice "Quiero pagar mi factura de mayo", el intent podría ser *pago_factura*



Conversación de principio a fin 1

Usuario: Hola, busco alojamiento en Málaga para agosto, somos 4 personas. ¿Qué me recomiendas?

Agente: ¡Hola! Un placer atenderle. Málaga en agosto es ideal. Para ayudarlo, ¿me indicaría las fechas exactas del viaje y si prefiere hotel o apartamento? Estoy a su entera disposición.

Usuario: Del 10 al 17 de agosto. Somos 2 adultos y 2 adolescentes. Buscamos hotel o resort con piscina y muy cerca de la playa. Calidad-precio razonable.

Llamada a la Herramienta:

JSON

```
google_hotels.search_hotels(  
  query="hotel playa piscina Málaga",  
  check_in_date="2026-08-10",  
  check_out_date="2026-08-17",  
  adults=4,  
  amenities=["pool", "beach_access"]  
)
```

Agente: Gracias. Le propongo el *Sol Guadalmar* (acceso directo a playa y piscinas) o el *Gran Hotel Miramar* (lujo frente al mar). ¿Le cotizo disponibilidad en alguno de ellos? ¡Un honor servirle!

Otra Conversación de principio a fin 2

Usuario: Hola, buenas tardes. Necesitamos reservar unas vacaciones en Málaga para este agosto. Somos 4 personas. ¿Qué opciones tienen?

Agente: ¡Hola! Será un absoluto honor ayudarle. Málaga en agosto es hermosa. Para afinar la búsqueda, ¿podría indicarme las fechas de entrada y salida y las edades de sus acompañantes?

Usuario: Viajo con mi pareja y mis dos hijos de 15 y 17 años. Queremos ir a mitad de mes, una semana entera.

Agente: Entendido, muchas gracias. Al ser adolescentes cuentan como adultos. Tomaré del lunes 10 al lunes 17 de agosto. ¿Prefiere hotel con piscina cerca de la playa o algo más céntrico?

Usuario: Preferimos hotel o resort con piscina y acceso directo a la playa. No tenemos un presupuesto fijo, pero buscamos una buena relación calidad-precio.

Llamada a la Herramienta:

JSON

```
google_hotels.search_hotels(  
  query="hotel resort playa piscina Málaga",  
  check_in_date="2026-08-10",  
  check_out_date="2026-08-17",  
  adults=4,  
  amenities=["pool", "beach_access"]  
)
```

Agente: Gracias por su paciencia. Encontré el *Sol Guadalmar* (excelente calidad-precio, pie de playa y piscinas) y el *Gran Hotel Miramar* (gran lujo frente al mar). ¿Cuál le atrae más?

Usuario: El Sol Guadalmar suena muy bien por la relación calidad-precio. ¿Tienen habitaciones con vistas al mar disponibles para nosotros?

Llamada a la Herramienta:.....

Agente Orientado a la Ejecución de Procesos

Los agentes que vimos tienen una interacción conversacional con el humano. Sin embargo puede haber sistemas agénticos que ejecuten procesos que hagan uso de LLM y una arquitectura agéntica.

Así estos agentes pueden ir ejecutando su ciclo iterativamente si reciben prompts o instrucciones de mejora más allá que los programados dentro de su sistema.

Elección del

Modelo de Lenguaje

¿Cómo elegir el modelo de lenguaje para un Agente?

Un agente puede usar varios modelos de lenguaje, de diferentes costos y características. Para elegirlo, se pueden tener en cuenta los siguientes puntos:

Dimensión	Consideraciones
Capacidades y Razonamiento	Profundidad del razonamiento(multi-step, abstract), complejidad sobre como sigue instrucciones, profundidad y síntesis del conocimiento y la sofisticación en el uso nativo de herramientas/llamadas a función
Tamaño del modelo	Trade-off entre modelos grandes de proposito general (capacidades amplias, costos más altos / mayor latencia) vs modelos más pequeños y especializados (eficiencia menor costo / latencia)
Tamaño de la ventana de contexto	Habilidad de procesar y retener la información, manejo de diálogos grandes, soporta de tareas multipaso, y capacidad de In-Context Learning
Uso de herramientas nativo / llamadas a función	Confiable para identificar que herramienta cuando y cómo debe usar, formateo de parámetros correcto, adherencia a schemas y facil de integrar con framework de agentes
Adaptabilidad y Fine-tuning	Fácil de hacer fine-tuning, performance de RAG y la capacidad de ICL
Especificidad acorde al Dominio o la tarea	Alineación del modelo con tareas específicas o la industria, potencial de especialización
Explicabilidad (XAI)	Herramientas y características de la LLM para entender las decisiones y salidas, importante para depurar, tener confianza y cumplimiento de normas

¿Cómo elegir el modelo de lenguaje para un Agente?

Dimensión	Consideraciones
Viabilidad Operacional	Latencia, throughput, costo computacional y la eficiencia total. Trade-off entre grandes y pequeños modelos especializados
Robustez y disponibilidad	resistencia a entradas adversas, performance consistente, exactitud, habilidad para detectar incertidumbres y la baja tendencia a alucinar
Seguridad	Mitigación de bias, filtros de seguridad, privacidad de datos, acceso seguro al modelo, y protección contra vulnerabilidades específicas del modelo.
Integración y despliegue	Fácil de integrar en sistemas existentes y compatibilidad de los entornos (cloud, on-premises o edge)
Mantenimiento y Gobernanza	Experiencia técnica requerida, soporte provisto, licencia, características de explicabilidad y el manejo continuo (AgentOps)
Costo	Costos de inferencia, presupuesto necesario para el fine-tuning, hosting, cargos por llamadas a la API y costo total de ownership
Licencia y términos de uso	Compatibilidad comercial y uso interno, ownership de los datos y las salidas y restricciones de uso
Soporte de proveedor y ecosistema	Calidad de la documentación, soporte técnico, roadmap del modelo y la disponibilidad de las herramientas y los recursos de la comunidad.
Privacidad de Datos y Seguridad operativa	Cómo manejan los datos el proveedor, la seguridad de modelos y datos de agentes propios

Eligiendo los modelos de lenguaje

Los modelos pueden ser categorizados en tres tipos principales:

Propietarios

Sus funcionalidades completas son solo accesibles mediante interfaces web o APIs pagas. Se desconocen detalles de su arquitectura interna.

Ejemplos:

GPT-5.6, Claude 5, Gemini 3.6

Open Models

Modelos de pesos abiertos ("open weights") que exponen públicamente su arquitectura y parámetros para su uso y personalización.

Ejemplos:

Gemma 3, Llama 3.3

Open-Source

Liberan por completo los datos de entrenamiento, el código de entrenamiento, los entornos de evaluación y los pesos del modelo.

Ejemplos:

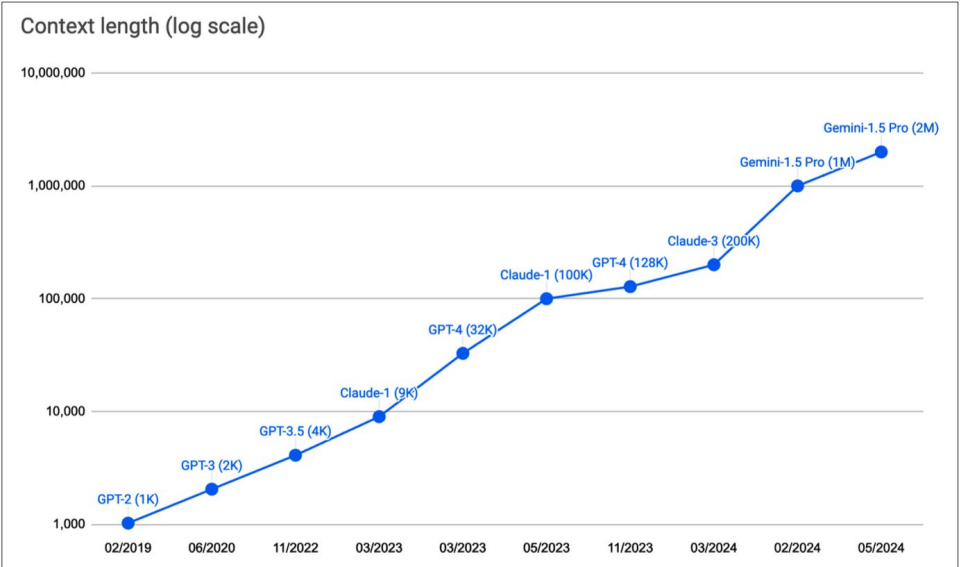
OLMo 3.1 (allenai.org/olmo)

Ventana Contexto

Se le llama Ventana de Contexto al máximo de información que puede ser incluido en un prompt.

El tamaño máximo viene dado por el modelo que usamos. Por ejemplo, GPTs tienen 1K, 2K y 4K tamaño de contexto respectivamente cada generación (1,2,3).

El crecimiento de la ventana de contexto se detuvo, teniendo ahora modelos de ultima generación como Gemini 3.6 con 1M de tokens.



Ventana Contexto - Datos interesantes - Lost in the Middle (2023)

Hay investigaciones que dicen que un modelo entiende mejor las instrucciones dadas al principio que en el medio del prompt (<https://arxiv.org/abs/2307.03172>).

Técnica Needle in the haystack (NIAH) significa poner un texto en el prompt y hacer que lo encuentre. En la investigación, era más probable que lo encuentre cuando estaba al principio.

Input Context

Write a high-quality answer for the given question using only the provided search results (some of which might be irrelevant).

Document [1](Title: Asian Americans in science and technology) Prize in physics for discovery of the subatomic particle J/ψ. Subrahmanyan Chandrasekhar shared...

Document [2](Title: List of Nobel laureates in Physics) The first Nobel Prize in Physics was awarded in 1901 to Wilhelm Conrad Röntgen, of Germany, who received...

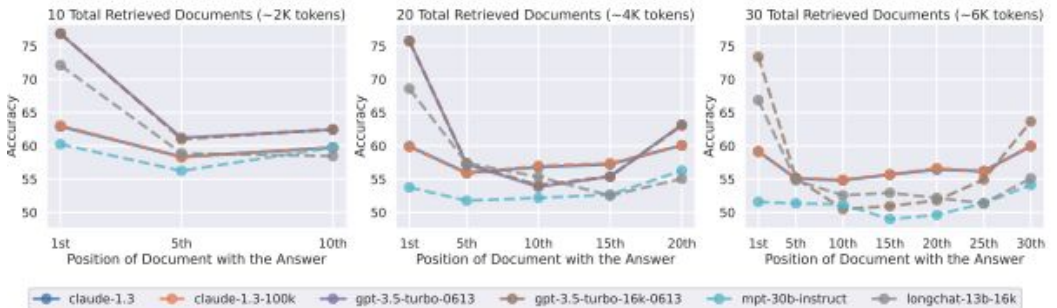
Document [3](Title: Scientist) and pursued through a unique method, was essentially in place. Ramón y Cajal won the Nobel Prize in 1906 for his remarkable...

Question: who got the first nobel prize in physics

Answer:

Desired Answer

Wilhelm Conrad Röntgen



Modelos propietarios

Desarrollador	Familia de Modelos	Modelos Destacados	Ventana de Contexto
OpenAI	GPT-5 / o3	GPT-5.2, o3-high	200K - 400K tokens
Google DeepMind	Gemini 3	Gemini 3.1 Pro, 3 Flash	1M - 10M tokens
Anthropic	Claude 4	Claude 4.7 Opus, 4.6 Sonnet	200K - 1M tokens
xAI	Grok-3 / Grok-4	Grok 4.1, Grok 3	131K+ tokens
Mistral AI	Mistral Large	Mistral Large 3	128K tokens
Moonshot AI	Kimi K2	Kimi K2.5, K2 Thinking	256K tokens

Model Card - Entiende los supuestos de los modelos

- Todos los modelos que usemos, debemos conocer la teoría que hay detrás. Hay muchos supuestos no escritos que acompañan al tipo de modelo e implementación. Utilizar el modelo desde una API es sencillo pero nuestro trabajo es conocer el detalle para garantizar un buen sistema de ML
- Las diferentes empresas suelen ofrecer mucha documentación en torno al modelo. Un concepto que está ganando fuerza es el de **Model Cards**.
- También es una buena práctica para nuestros modelos.
- Ejemplos:
 - [MedLM Model Card](#)
 - [GPT 4 System Card](#)
 - [Gemma 2 Model Card](#)
 - [Imagen 3 Model Card in Paper](#)
 - Extra - documentación de modelos de Gemini API: <https://ai.google.dev/gemini-api/docs/models>
- Datos que suelen incluir: descripción del modelo, descripción de los datos y el entrenamiento, riesgos, consideraciones éticas de AI, metodología de evaluación. Algunos incluso incluyen el impacto en la sociedad y evaluaciones hechas por 3rd parties.
- Ya sea haciendo nuestro modelo o usando un modelo pre-entrenado es interesante contar con estas cartas de modelo. También es interesante incluirlas en los proyecto de ML que tengamos de la facultad, el laboratorio o la empresa.

Alucinaciones y Bias

- Las alucinaciones se producen cuando el texto generado no es correcto o no esta basado en la realidad.
- Los factores que influyen son:
 - los datasets usados en el entrenamiento no tienen información suficiente para responder correctamente
 - los datos de entrenamiento pueden tener contenido ficticio o subjetivo, incluyendo opiniones y creencias
 - los datos necesarios para responder se pueden perder entre la masividad de los datos y optar por respuestas incorrectas

Consecuencias de las Alucinaciones

- Pueden llevar a la desinformación
- Existe el riesgo que individuos hayan manipulado y diseminado información falsa, validando narrativas incorrectas
- Podría haber vías hacia lenguaje ofensivo, privacidad o éticos
- Podría tener comportamientos discriminatorios de género, raza, religión or etnia producto de ser entrenados con datos con bias

Técnicas para reducir las alucinaciones

- Ajustar los parámetros de generación de texto
 - **Temperatura**
 - **Top-k Sampling**
 - **Top-p Sampling**
- Utilizar documentos externos con arquitecturas “retriever”
- Fine-Tuning LLMs

Alucinaciones: Control de Temperatura en LLMs

¿Qué es la Temperatura?

Los logits de salida son divididos por el parámetro de temperatura antes de aplicar la función softmax que calcula las probabilidades de cada palabra.

- **Temperatura Alta (ej. > 1)**

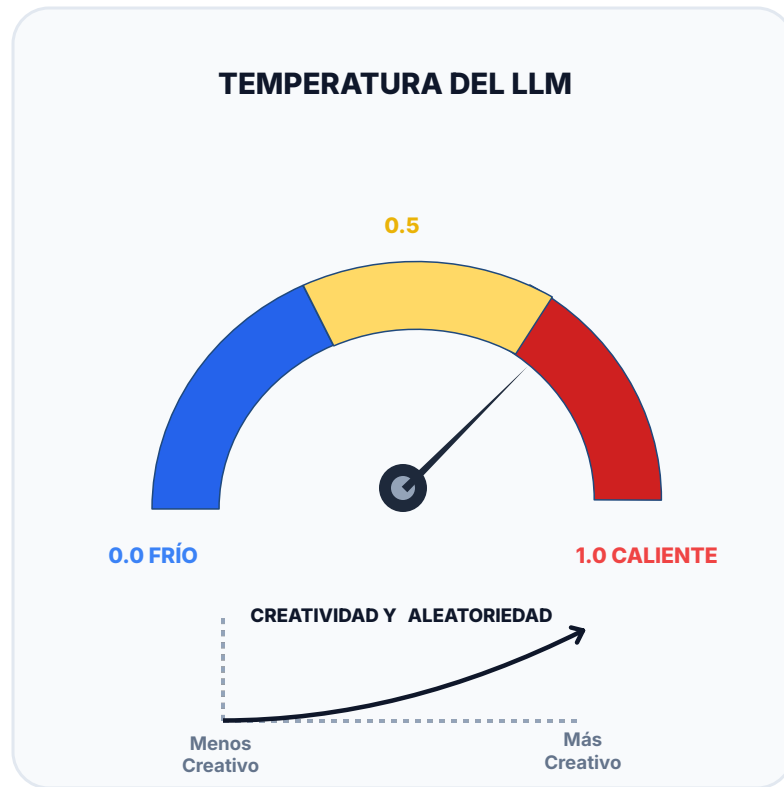
Vuelve la generación de texto mucho más creativa, variada y sorprendente al aplanar la distribución de probabilidad.

- **Temperatura Baja (ej. < 1)**

Hace que el modelo elija las palabras más probables, resultando en respuestas más exactas, coherentes y conservadoras.

- **Temperatura = 1**

Mantiene los logits originales sin escalar, utilizando la distribución de probabilidad natural entrenada del modelo.



Temperatura

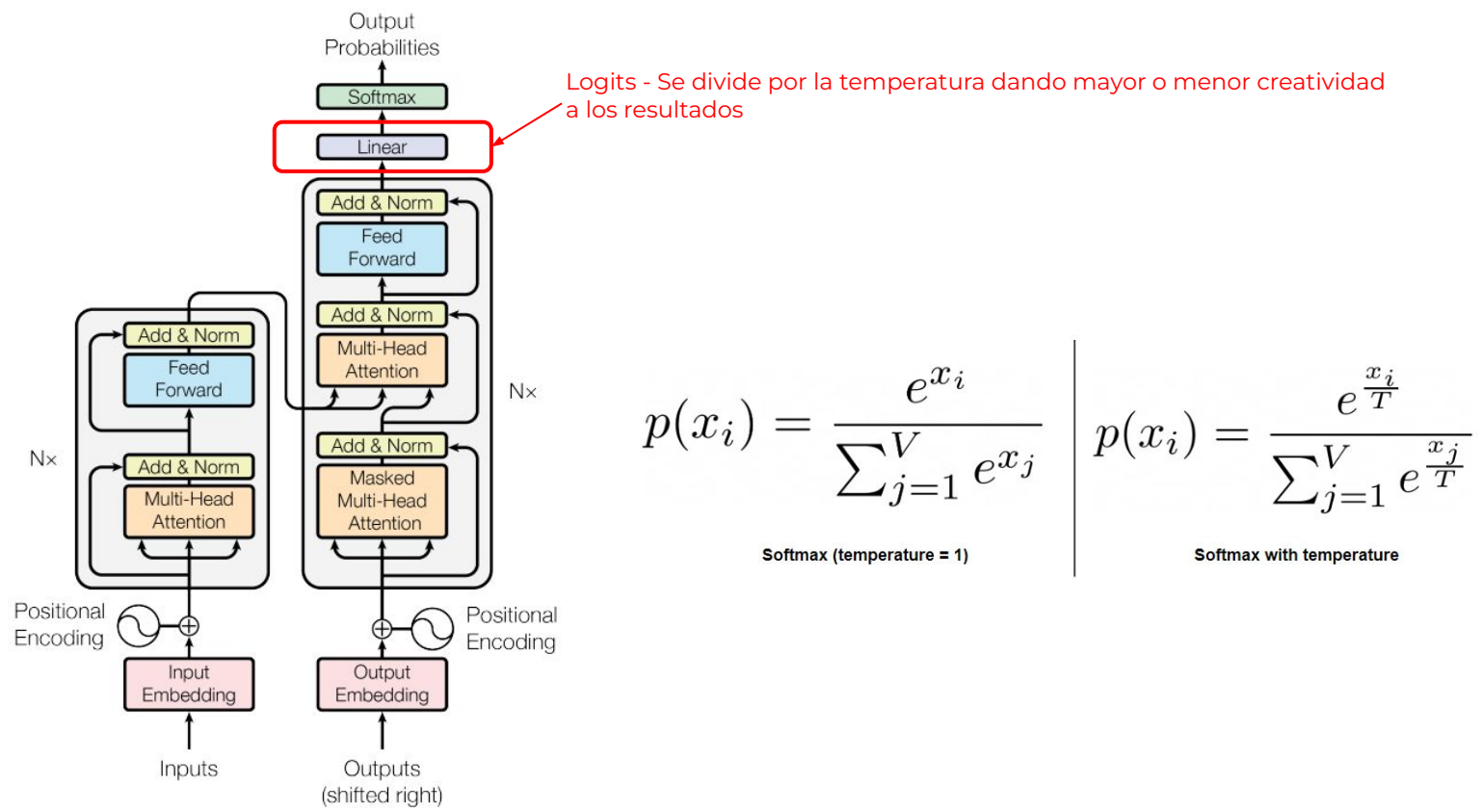
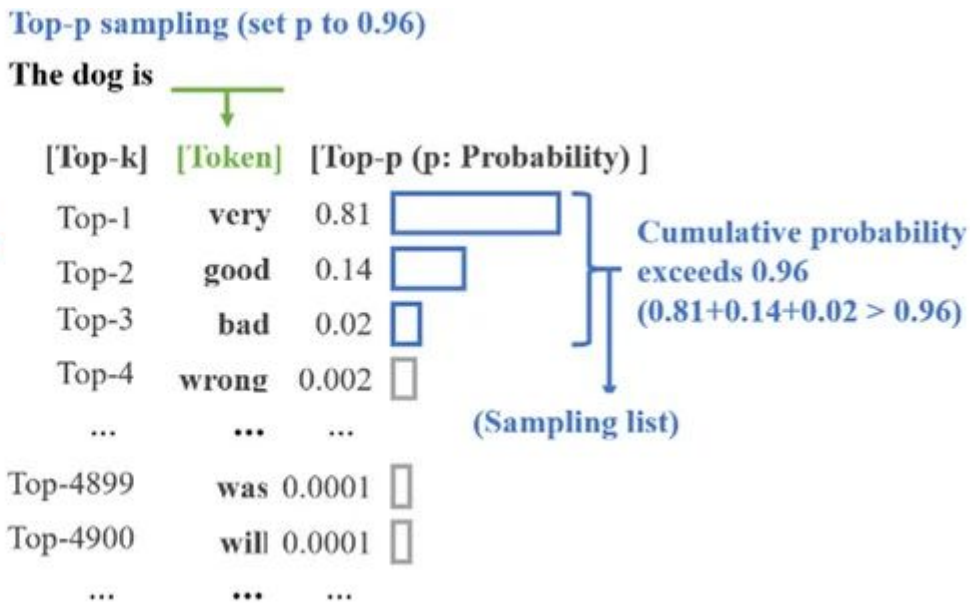
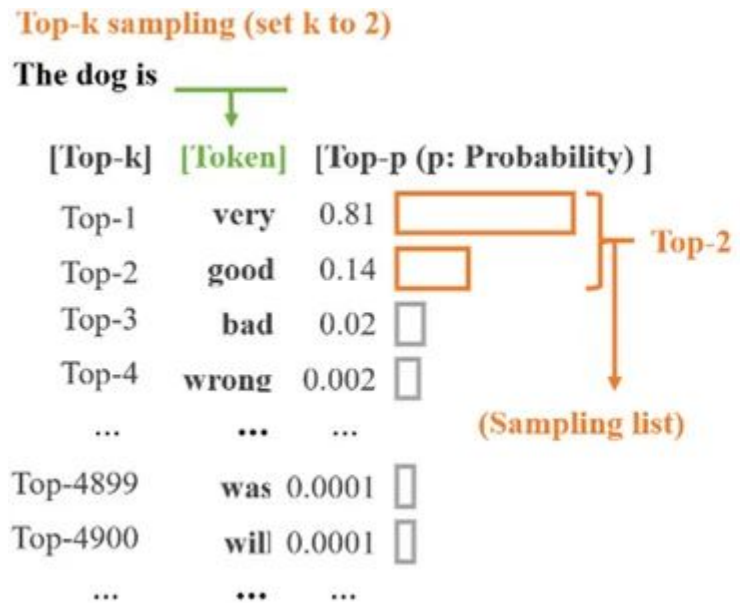


Figure 1: The Transformer - model architecture.

Alucinaciones: Top-p y Top-k Sampling

Top-K limita la selección a las K palabras más probables

Top-P limita la selección a las palabras con probabilidad acumulada mayor que P



Gráfica del Paper: <https://link.springer.com/article/10.1007/s10489-023-05201-3>

Notebook

Parámetros y contexto

La falsa idea del determinismo

Meld

Text Filters

File 1

File 2

--- TEMPERATURA 0.0 (Poca creatividad, analítico, repetitivo si se ejecuta varias veces)

Imagina que tienes varias cosas que quieres recordar o agrupar de alguna manera, como

En su esencia más simple, una **lista** es una **colección ordenada de elementos**.

Vamos a desglosarlo:

1. **Colección de elementos:** Significa que agrupa varias "cosas" o "items". Estas cosas
* **Ejemplo:** En una lista de compras, los elementos son "leche", "pan", "huevos"

2. **Ordenada:** Esto es clave. Los elementos en una lista tienen una posición específica
* **Ejemplo:** En una lista de pasos para una receta, el "paso 1" no es lo mismo que el "paso 2"

3. **Puede contener elementos repetidos:** A diferencia de otros tipos de colecciones
* **Ejemplo:** Tu lista de compras podría tener "manzanas" y luego más abajo "manzanas"

4. **Flexible/Dinámica:** Generalmente, puedes añadir nuevos elementos a una lista, que

--- TEMPERATURA 0.0 (Poca creatividad, analítico, repetitivo si se ejecuta varias veces)

Imagina que necesitas recordar varias cosas, como las compras que tienes que hacer, la

En esencia, una **lista** es una **colección ordenada de elementos**. Es una forma de

Aquí te detallo sus características principales:

1. **Colección de Elementos:** Contiene múltiples "cosas" (items, datos, objetos). Cada elemento tiene una posición específica.
2. **Orden Específico:** Los elementos en una lista tienen una posición. Hay un primer elemento, un segundo, etc.
3. **Puede Contener Diferentes Tipos:** Dependiendo del contexto (especialmente en programación), una lista puede contener diferentes tipos de datos.
4. **Permite Repeticiones:** Puedes tener el mismo elemento varias veces dentro de una lista.
5. **Modificable (Generalmente):** En la mayoría de los casos, las listas son dinámicas y permiten:
* **Añadir** nuevos elementos.
* **Quitar** elementos existentes.
* **Modificar** el valor de un elemento.
* **Reordenar** los elementos.

Ejemplos Cotidianos de Listas:

* **Lista de compras:** Leche, pan, huevos, frutas.
* **Lista de tareas pendientes (To-do list):** Pagar facturas, llamar al médico, enviar correos.
* **Lista de invitados a una fiesta:** Juan, María, Pedro, Ana.
* **Lista de ingredientes para una receta:** Harina, azúcar, huevos, levadura.

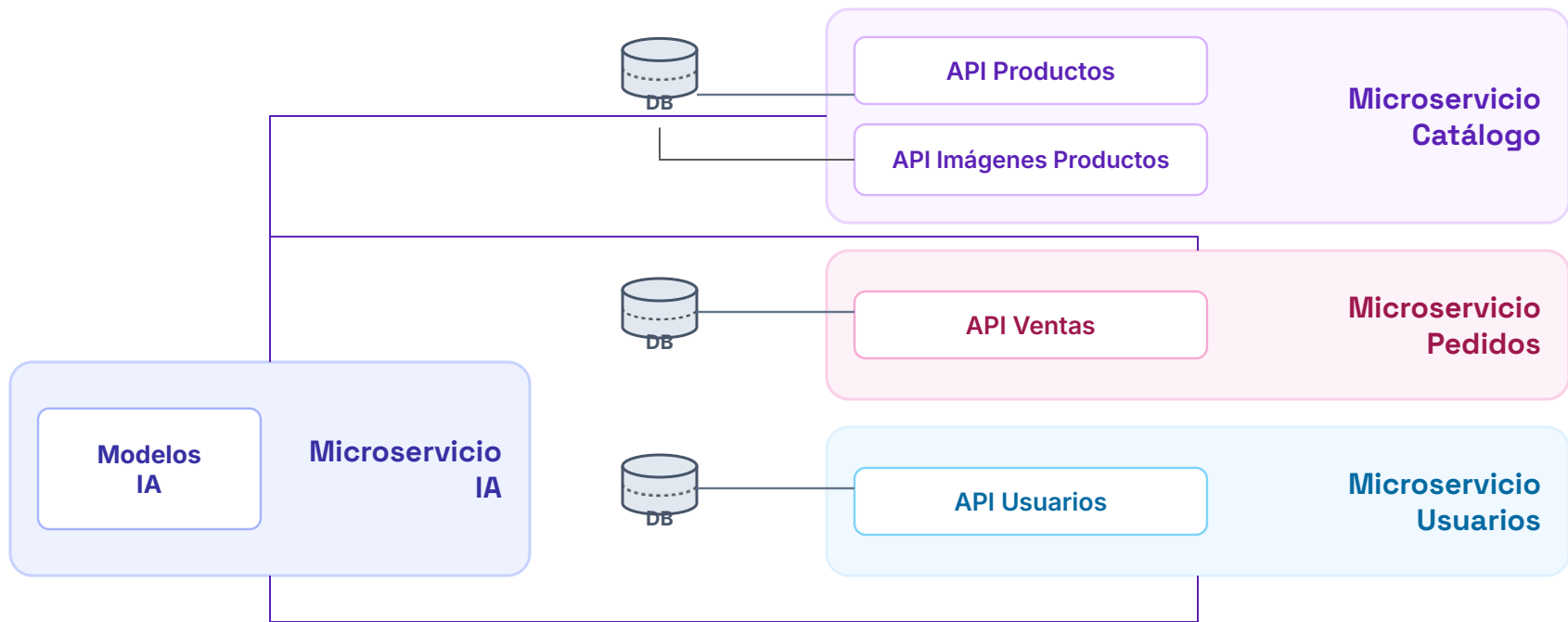
Julio 2026

ECI 2026: Escuela de Ciencias Informáticas - UBA - Argentina

Prof. Leticia L. Rodríguez - TEGAI

Arquitectura de la Solución y Herramientas

Microservicio y APIs



Arquitectura simplificada que muestra cómo se sirven modelos de IA a diferentes microservicios

Los estándares: API REST

REST API en Acción



Los estándares: API REST

Endpoint	<code>https://apiurl.com/review/new</code>
HTTP Method	<code>POST</code>
HTTP Headers	<code>content-type: application/json</code> <code>accept: application/json</code> <code>authorization: Basic abase64string</code>
Body	<pre>{ "review" : { "title" : "Great article!", "description" : "So easy to follow.", "rating" : 5 } }</pre>

Métodos HTTP

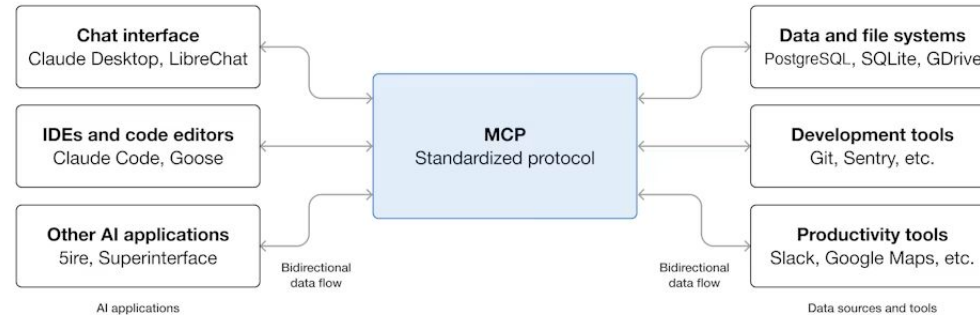
- POST - Crear
- UPDATE - Actualizar
- DELETE - Borrar
- GET - Obtener

MCP - Model Context Protocol

Tanto Google ADK, como otros frameworks para crear agentes permiten usar herramientas que cumplan el protocolo MCP.

MCP es un estándar abierto diseñado para que cualquier LLM pueda hacer uso de herramientas que están creadas y hospedadas en diferentes partes.

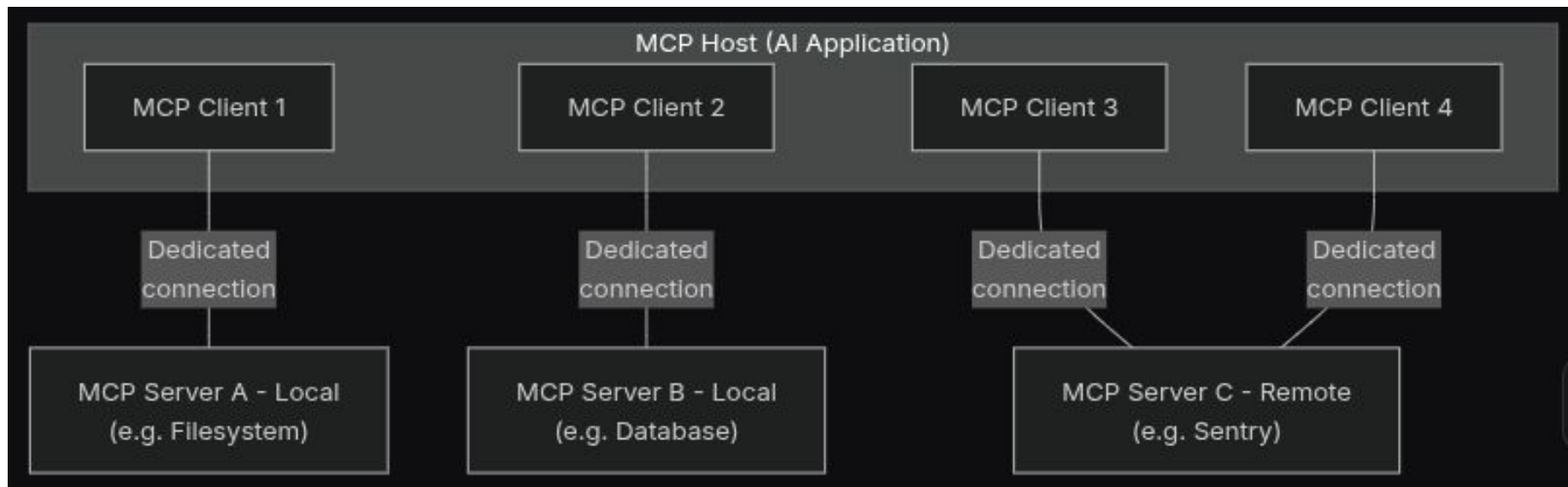
La ventaja es que permite reutilizar herramientas y estandariza la comunicación entre el agente y las herramientas externas.



MCP <https://modelcontextprotocol.io/docs/getting-started/intro>

MCP - Model Context Protocol - Arquitectura

- **Host de MCP:** La aplicación de IA que utiliza uno o varios clientes de MCP, para acceder a herramientas distintas.
- **Ciente de MCP:** Un componente que mantiene una conexión con un servidor de MCP y obtiene contexto de dicho servidor para que lo utilice el host de MCP (la aplicación / Agente)
- **Servidor de MCP:** Un programa que sirve a los clientes de MCP. Da el contexto y tiene todo lo necesario para que funcione la herramienta.



MCP <https://modelcontextprotocol.io/docs/getting-started/intro>

Demo

Herramientas - MCP

Herramientas provistas APIs

	Asana Manage projects, tasks, and goals for team collaboration		Atlassian Manage issues, search pages, and update team content		Google Search Perform web searches using Google Search with Gemini		Google Cloud Storage Access and perform operations on Google Cloud Storage buckets and objects
	Hugging Face Access models, datasets, research papers, and AI tools		Grafana Cloud Query metrics, logs, traces, and manage dashboards, alerts, and incidents from Grafana Cloud		GitHub Analyze code, manage issues and PRs, and automate workflows		MLflow AI Gateway Route LLM requests for multi-provider access with built-in governance
	GitLab Perform semantic code search, inspect pipelines, manage merge requests		AG-UI Build interactive chat UIs with streaming, state sync, and agentic actions		MLflow Scorers Use ADK evaluators as MLflow scorers for agent evaluation		Paypal Manage payments, send invoices, and handle subscriptions

Google ADK Integraciones (algunas): <https://adk.dev/integrations/>
Herramientas APIs <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>

Herramientas provistas APIs

Básicamente, el ADK te simplifica un montón el trabajo con APIs REST externas. Lo que hace es tomar la especificación OpenAPI (v3.x) y generarte automáticamente las *tools* que podés invocar. Así te ahorrás todo el laburo manual de tener que codear una función distinta por cada *endpoint* de la API.

```
openapi_spec_string = """
{
  "openapi": "3.0.0",
  "info": {
    "title": "Simple Pet Store API (Mock)",
    "version": "1.0.1",
    "description": "An API to manage pets in a store, using httpbin for responses."
  },
  "servers": [
    {
      "url": "https://httpbin.org",
      "description": "Mock server (httpbin.org)"
    }
  ],
  "paths": {
    "/get": {
      "get": {
        "summary": "List all pets (Simulated)",
        "operationId": "listPets",
        "description": "Simulates returning a list of pets. Uses httpbin's /get endpoint which echoes query parameters.",
        "parameters": [
          {
            "name": "limit",
            "in": "query",
            "description": "Maximum number of pets to return",
            "required": false,
            "schema": { "type": "integer", "format": "int32" }
          }
        ],
        {
          "name": "status",
          "in": "query",
          "description": "Filter pets by status",
          "required": false,
          "schema": { "type": "string", "enum": ["available", "pending", "sold"] }
        }
      ],
      "responses": {
        "200": {
          "description": "A list of pets (echoed query params).",
          "content": { "application/json": { "schema": { "type": "object" } } }
        }
      }
    }
  }
}
```

Ejemplo de Json OpenAPI spec

Más información sobre OpenAPI: <https://www.ibm.com/think/topics/open-api>

1. **Obtain Spec:** Get your OpenAPI specification document (e.g., load from a `.json` or `.yaml` file, fetch from a URL).
2. **Instantiate Toolset:** Create an `OpenAPIToolset` instance, passing the spec content and type (`spec_str / spec_dict, spec_str_type`). Provide authentication details (`auth_scheme, auth_credential`) if required by the API.

```
from google.adk.tools.openapi_tool.openapi_spec_parser.openapi_toolset import OpenAPIToolset

# Example with a JSON string
openapi_spec_json = '...' # Your OpenAPI JSON string
toolset = OpenAPIToolset(spec_str=openapi_spec_json, spec_str_type="json")

# Example with a dictionary
# openapi_spec_dict = {...} # Your OpenAPI spec as a dict
# toolset = OpenAPIToolset(spec_dict=openapi_spec_dict)
```

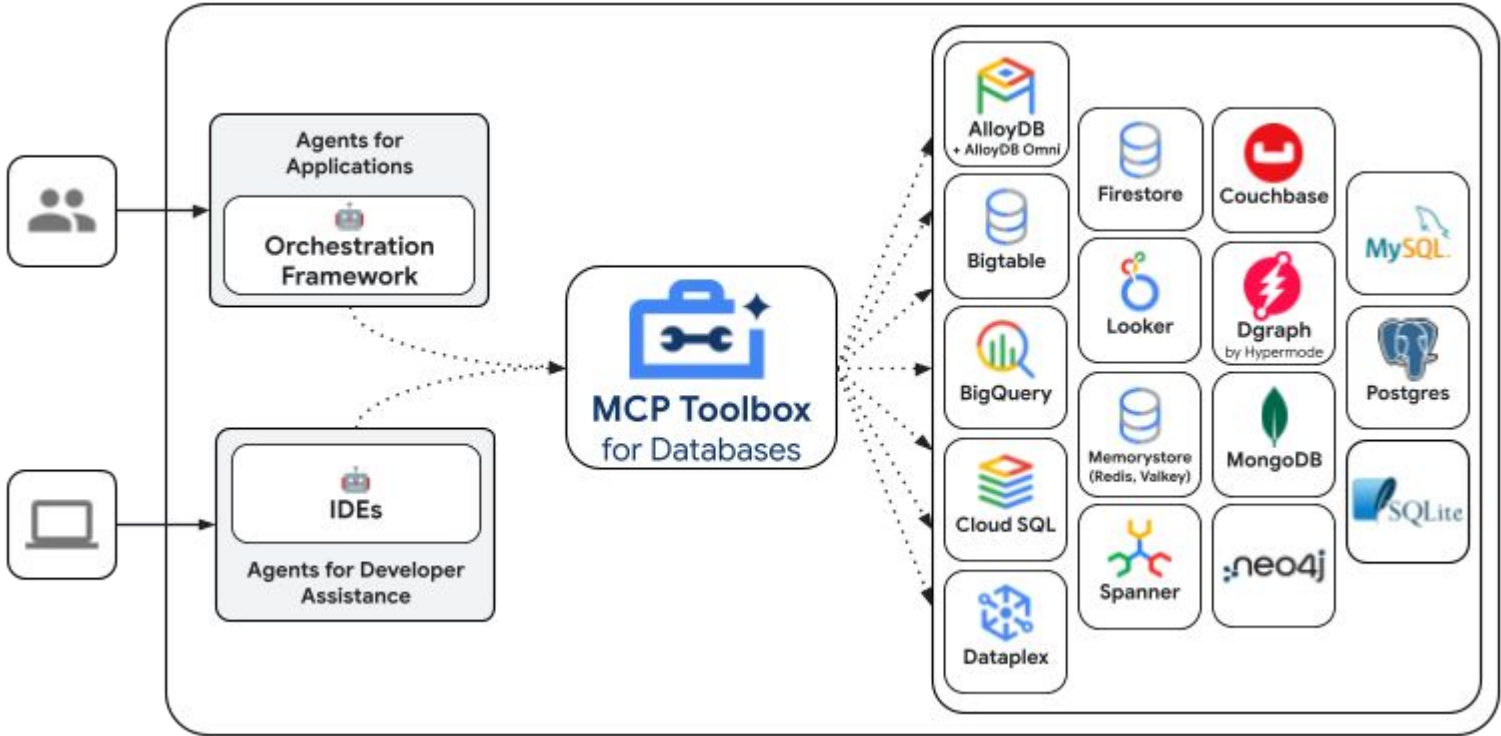
3. **Add to Agent:** Include the retrieved tools in your `LlmAgent`'s `tools` list.

```
from google.adk.agents import LlmAgent

my_agent = LlmAgent(
    name="api_interacting_agent",
    model="gemini-flash-latest", # Or your preferred model
    tools=[toolset], # Pass the toolset
    # ... other agent config ...
)
```

<https://adk.dev/tools-custom/openapi-tools/>

MCP Toolbox para bases de datos en Google ADK



Documentación: <https://adk.dev/integrations/mcp-toolbox-for-databases/>

Comparativa: Herramienta, Agente como Herramienta, Subagente y MCP

Concepto	Qué es	Propósito Principal	Cuándo usarlo
Herramienta Tradicional (Tool)	Función atómica, determinista o API (ej. código, SQL, cálculo).	Resolver una tarea puntual y directa sin razonamiento intermedio.	Para acciones predecibles y de un solo paso (ej. calcular un valor, buscar en una base de datos).
Agente como Herramienta (Agent-as-a-Tool)	Un agente autónomo encapsulado detrás de un esquema de herramienta estándar.	Delegar un subproblema complejo a una caja negra que razona antes de responder.	Cuando necesitas que un bloque ejecute un bucle autónomo y multi-paso antes de devolver el resultado final.
Subagente	Una entidad con rol específico dentro de un sistema multi-agente jerárquico.	Dividir responsabilidades en un equipo o árbol de trabajo coordinado.	Cuando diseñas flujos complejos (<i>Supervisor-Worker</i>) donde varios agentes comparten un mismo marco de orquestación.
MCP (Model Context Protocol)	Un protocolo de comunicación estándar (el "USB-C" de la IA) para exponer datos y herramientas.	Estandarizar, desacoplar y asegurar el acceso de los LLMs a la infraestructura.	Cuando quieres reutilizar conectores, manejar recursos masivos de solo lectura o aislar la seguridad de tus sistemas.

Public MCP Servers - Original de Anthropic MCP

registry.modelcontextprotocol.io?q=microsoft

search Riva New Tab Chat | Microso... Partner Center List of Proceed... Outlook LevelUp for Mi... TEGAI - Google... Legal info Halt and Catch... CAS - Login ce...

microsoft

☒ Show only latest versions

com.microsoft.esrp/esrp-oss-mcp-testv26.226.135743

All Azure MCP tools to create a seamless connection between AI agents and Azure services.

4/29/2026

com.microsoft/esrp-oss-mcp-testv26.527.121158

The ESRP OSS MCP server exposes tools to discover & validate trusted Microsoft OSS Packages.

5/27/2026

com.microsoft/microsoft-fabricv1.2.0

MCP tools for interacting with Microsoft Fabric

7/8/2026

com.microsoft/microsoft-learn-mcpv1.0.0

Official Microsoft Learn MCP Server – real-time, trusted docs & code samples for AI and LLMs.

11/21/2025

com.microsoft/nugetv1.4.16

A Model Context Protocol (MCP) server for NuGet.

6/30/2026

com.microsoft/powerbi-modeling-mcpv0.5.0-beta.11

The Power BI Modeling MCP Server brings Power BI semantic modeling capabilities to your AI agents.

6/25/2026

com.microsoft/sentinel-data-explorationv1.0.1

Find relevant security data from Sentinel data lake for building effective agents. More:aka.ms/s/de

1/15/2026

com.microsoft/template-server-namev0.0.12-alpha.5932986

Template MCP server example.

2/26/2026

com.microsoft/workiq-admintoolsv1.0.0

MCP Server containing tools relating to Microsoft Admin Center

3/25/2026

com.microsoft/workiq-calendartoolsv1.0.1

Calendar tools for creating, updating, deleting events, managing invites, and checking availability.

7/10/2026

com.microsoft/workiq-m365copilotv1.0.0

Search across M365 content including documents, emails, sites, files, and chats.

3/25/2026

com.microsoft/workiq-mailtoolsv1.0.1

Email tools for creating, sending, replying, updating, deleting, and searching messages.

3/25/2026

com.microsoft/workiq-meserverv1.0.0

Tools for retrieving user details, manager, team or direct reportees from Microsoft Graph.

3/25/2026

com.microsoft/workiq-odspremoteserverv1.0.0

OneDrive and Sharepoint Remote MCP Server

3/25/2026

<https://registry.modelcontextprotocol.io/>
Github: <https://github.com/modelcontextprotocol/servers>

Public MCP Servers

MCP Registry

Type to search



Connect models to the real world

Servers and tools from the community that connect models to files, APIs, databases, and more.

Search MCPs

All MCP servers 142



Markitdown

Install

Convert various file formats (PDF, Word, Excel, images, audio) to Markdown.

By microsoft ☆ 164,668



Netdata

Install

Real-time infrastructure monitoring with metrics, logs, alerts, and ML-based anomaly detection.

By netdata ☆ 79,586



Context7

Install

Up-to-date code docs for any prompt

By upstash ☆ 58,890



Chrome DevTools MCP

Install

MCP server for Chrome DevTools

By ChromeDevTools ☆ 46,637



Playwright

Install

Automate web browsers using accessibility trees for testing and data extraction.

By microsoft ☆ 34,931



GitHub

Install

Connect AI assistants to GitHub - manage repos, issues, PRs, and workflows through natural language.

By github ☆ 31,346

<https://github.com/mcp?page=1>

Patrones para Agentes



Así como en los años 2000 empezaron a surgir patrones de diseño: Business Object, Data Access Object, Factory, etc. Dentro de la teoría de agentes, algunos autores hablan de patrones.

<https://www.langchain.com/blog/choosing-the-right-multi-agent-architecture>

<https://claude.com/blog/multi-agent-coordination-patterns>

<https://resources.anthropic.com/hubfs/Building%20Effective%20AI%20Agents-%20Architecture%20Patterns%20and%20Implementation%20Frameworks.pdf>

<https://developers.googleblog.com/developers-guide-to-multi-agent-patterns-in-adk/>

<https://machinelearningmastery.com/7-must-know-agentic-ai-design-patterns/>

Niveles de Maduración de Agentes

A mayor autonomía y complejidad del sistema agéntico, mayores son los riesgos y controles requeridos. Clasificamos la evolución y madurez de los agentes en 6 niveles clave:

1

Sistema Básico Agéntico

Single agent

2

Workflows de Agentes

Dynamic workflow agents

3

Patrones Introspectivos

React and reflexion agents

4

Sistemas Multi-Agénticos

Multi-agent systems

5

Multi-Agentes Avanzados

Advanced multi-agent with meta-agents

6

Feedback y Aprendizaje

Self-correcting agents

Nivel 1: Sistema básico agéntico (single agent)

single agent

tools

semi-autónomo

Agentes simples que pueden manejar tareas bien definidas y específicas de manera semi-autónoma, usando workflows simples y definidos y llamadas función para tools externas.

Escalabilidad: Adaptable pero rígido. Los flujos de trabajo son rígidos, poco espacio para innovar.

Complicaciones: Precisa las tareas bien definidas y minimiza los riesgos de violaciones de las políticas

Implementación (Patrones):

- Single-Agent
- Llamada a función estática (Static Function Calling)
- Watchdog Timeout
- Agente llama a Humano (Agent Calls Human)

Nivel 2: Sistema dinámico de workflows de Agentes (dynamic workflow agents)

single agent

dynamic tools

semi-autónomo

Un agente que puede dinámicamente seleccionar sobre una variedad de herramientas o APIs para resolver un problema, dándole mayor flexibilidad.

Escalabilidad: más versátiles y eficientes, pueden atacar una variedad de problemas complejos eligiendo la herramienta adecuada

Complicaciones: son manejables si se seleccionan bien las herramientas que pueden usar pero requiere monitoreo sobre su comportamiento a medida que crece su autonomía.

Implementación (Patrones/Técnicas):

- Agent Router (Básico)
- Dynamic Tool Selection
- Simple RAG
- Simple Retry

Nivel 3: Sistema agéntico con patrones introspectivos como React y Reflexion

single agents

feedback loops

self-correction

Agentes simples que usan métodos como React y patrones de reflexión para incorporar razonamiento paso a paso y auto-reflexión (self-reflection). Esto permite que aprendan de sus acciones y se corrijan a través de ciclos de feedback (feedback loops)

Escalabilidad: la introducción de feedback y autocorrección permite al agente tomar tareas más complicadas y mejorarse a través del tiempo, creando caminos para escalar.

Complicaciones: requieren monitoreo en tiempo real y pruebas de correctitud que se vuelven esenciales para asegurar que el agente se sigue alineando a las políticas mientras aprende

Implementación (Patrones):

- React
- Reflexion
- Instruction Fidelity Auditing (Auditoria de fidelidad de instrucciones)
- Reintento Adaptativo con mutación de prompt (Adaptive Retry with Prompt Mutation)

Supervisor Watchdog con Timeout

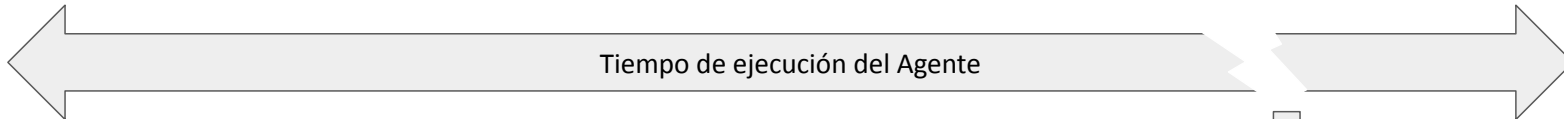
Este patrón encierra el agente en un contador de tiempo. Si el agente no completa la tarea en el tiempo determinado, se cancela generando un fallback que puede ser un logueo, escalar a un humano o usar otro agente.

Puede pasar si un agente se freezea o entra en loop infinito

Inicio



Tiempo expirado



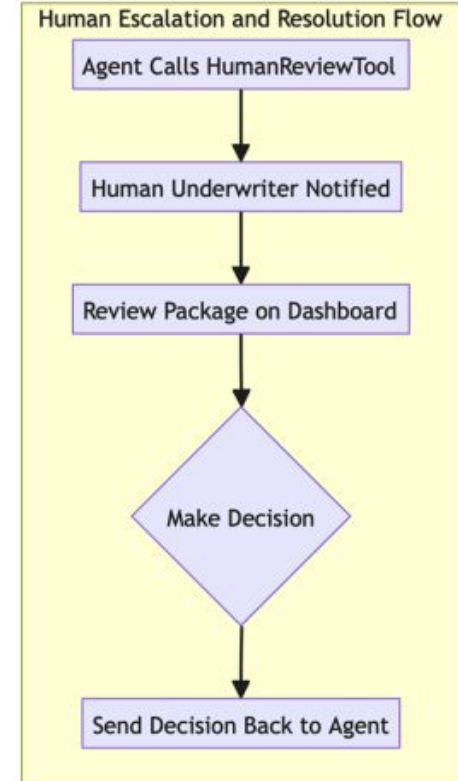
Fallback

Agente llama a Humano (Human in the Loop)

Permite hacer una escalación formal a un humano.

Cuando el agente encuentra una situación que requiere de la intervención de un humano, guarda su contexto y manda toda la información relevante al humano para que pueda tomar una decisión.

Mientras, el agente frena su flujo de trabajo. Cuando el humano aprueba toma la decisión, el agente continua con la información o validación provista.



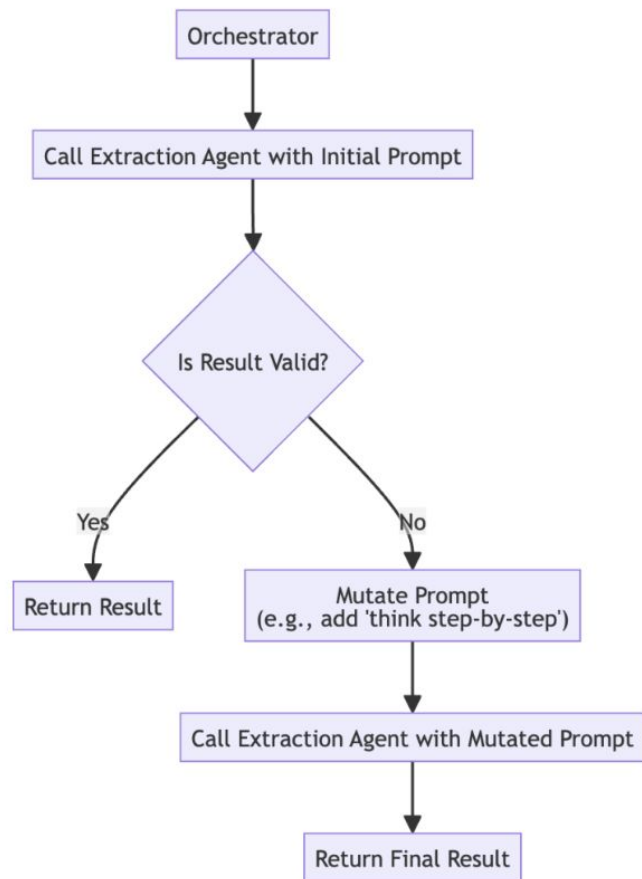
Adaptive Retry with Prompt Mutation

Implementa reintentos inteligentes y adaptativos. Después de que haya fallado enviar el pedido al agente, se modifica o muta el prompt y se vuelve a intentar con el objetivo de obtener la respuesta adecuada.

Se puede:

- Refrasear el prompt
- Agregar ejemplos
- Descomponer: pedir que use CoT u otras técnicas que lo ayuden a pensar.
- Que se endurezcan las restricciones (respondé en un json válido)

With Model Mutation: Sugiero por un tema de costos también implementarlo cambiar las versiones de los modelos, es decir, pasar de un flash a un pro en modelos Gemini por ejemplo.

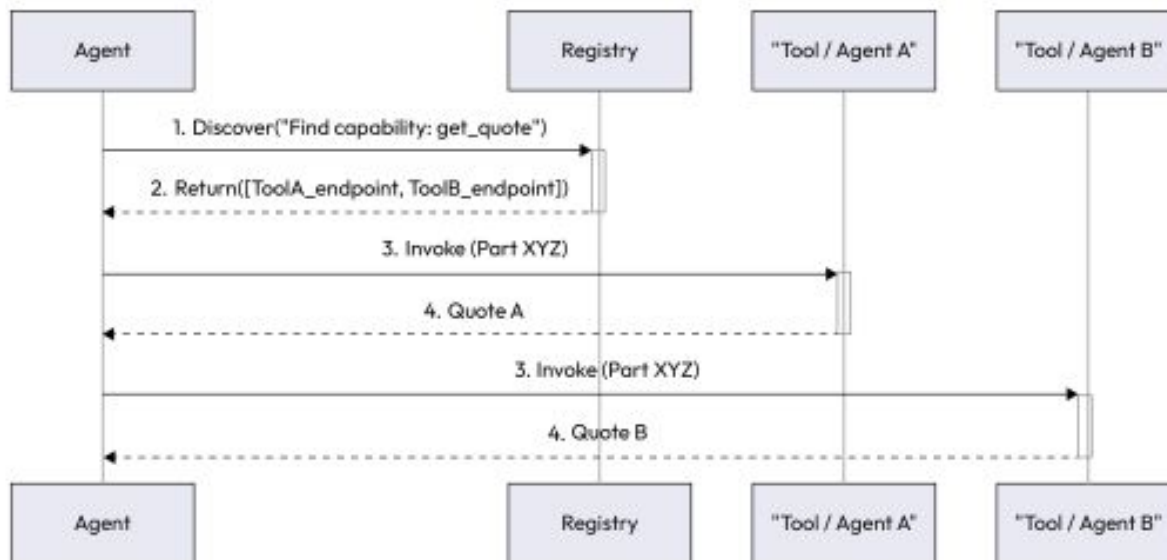


Registro Herramientas / Agentes

El patrón de **Registro Herramientas / Agentes** funciona como unas "páginas amarillas" dinámicas:

1. **Registro:** Las herramientas y agentes publican sus metadatos y *endpoints* al ser desplegados.
2. **Descubrimiento:** Los agentes buscan las capacidades que necesitan (ej. "reservar vuelo") sin usar código rígido (*hardcoded*).
3. **Invocación:** El registro devuelve la dirección exacta y el agente llama al servicio.

Debe ser muy cuidadoso al usar registros publicos, dado las vulnerabilidades de Seguridad que veremos sobre Herramientas.

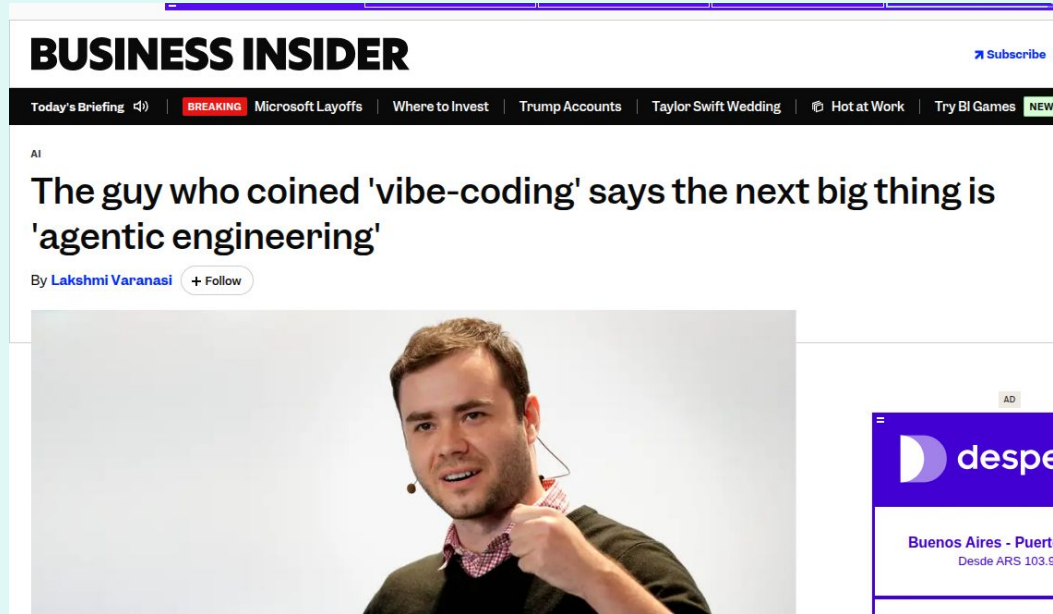


Diseño técnico del Agente y Arquitectura Multiagente

Las preguntas fundamentales desde el punto de vista técnico que tenemos que hacernos son:

- ¿Multi-agente? Una de las preguntas fundamentales para hacernos es si va a ser un sistema de 1 agente o multi-agente. Generalmente, por un tema de modularidad, será multiagente.
- ¿Patrones? Patrones a aplicar, existen infinitudes para distintos fines y además está la creatividad.
- ¿Herramientas? También, que herramientas vamos usar y acá tendremos que evaluar
- ¿MCP? : ¿Qué herramientas podemos utilizar?
- ¿A2A? : ¿Que otros agentes podemos reutilizar?
- ¿Memoria?: ¿Qué datos va a precisar almacenar el agente y cómo debe hacerlo?
- ¿Usuarios?: ¿Quienes van a ser? ¿Cómo van a acceder - interfaz? ¿Qué datos se van a almacenar y como cuidar su privacidad?
- ¿Responsabilidad?: Temas de IA Responsable, Safety. Incluir la seguridad y responsabilidad en IA en todo lo que hagamos.
- ¿Gobernanza?: ¿Cómo publicar nuestro agente?

Agentic Engineering



<https://www.businessinsider.com/agentic-engineering-andrej-karpathy-vibe-coding-2026-2>

Agentic Engineering: Factory Model

Se visualiza la creación de fábricas de software donde el código este automatizado por Agentes.

La función de los desarrolladores seria:

- Definir las especificaciones
- Diseñar las medidas de seguridad
- Revisar y Aprobar

Muchas Software Factory están envisionando la construcción de estos pipelines que requieren más complejidad que la vista.

¿Cómo se hace el seguimiento? ¿Cómo se diseña la fabrica?

Los desarrolladores tendrán también una función de arquitecto en armar todos estos pipelines automatizados.

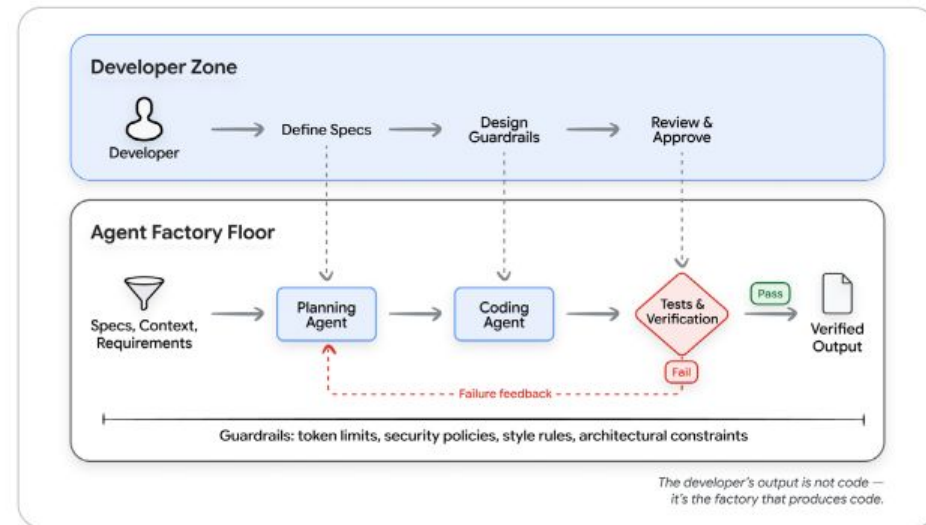
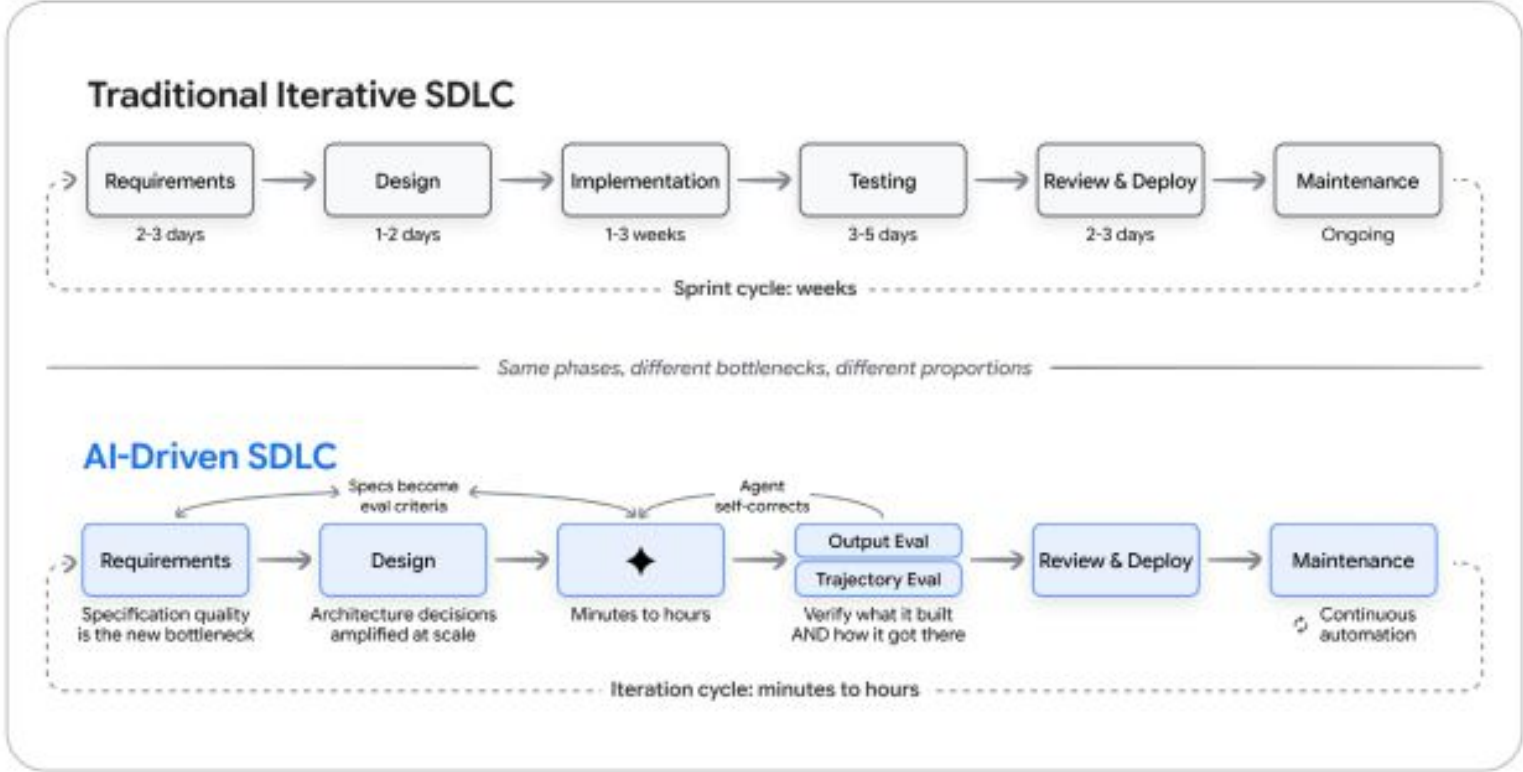


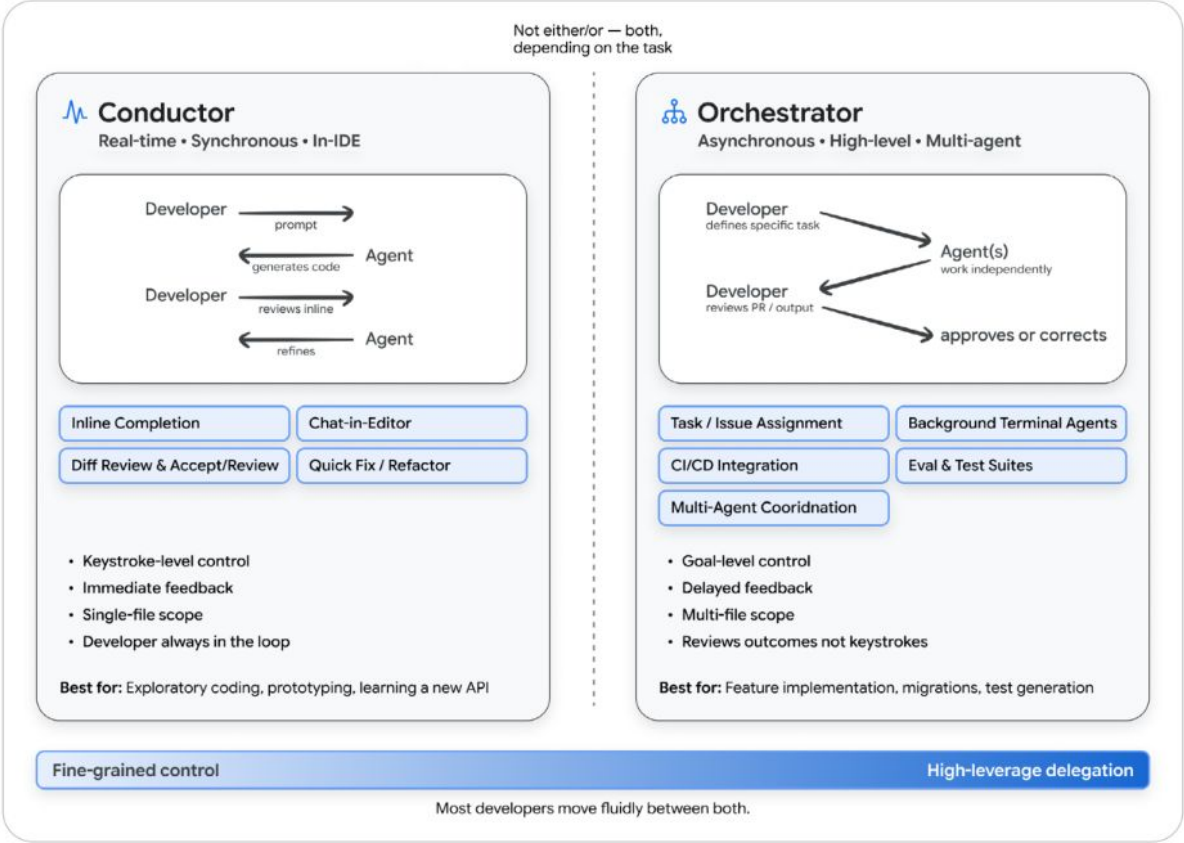
Figure 6: The Factory Model Developer designs the system -> agents produce the code -> tests verify the output.

Agentic Engineering: Redefinición del Ciclo de Vida del Software

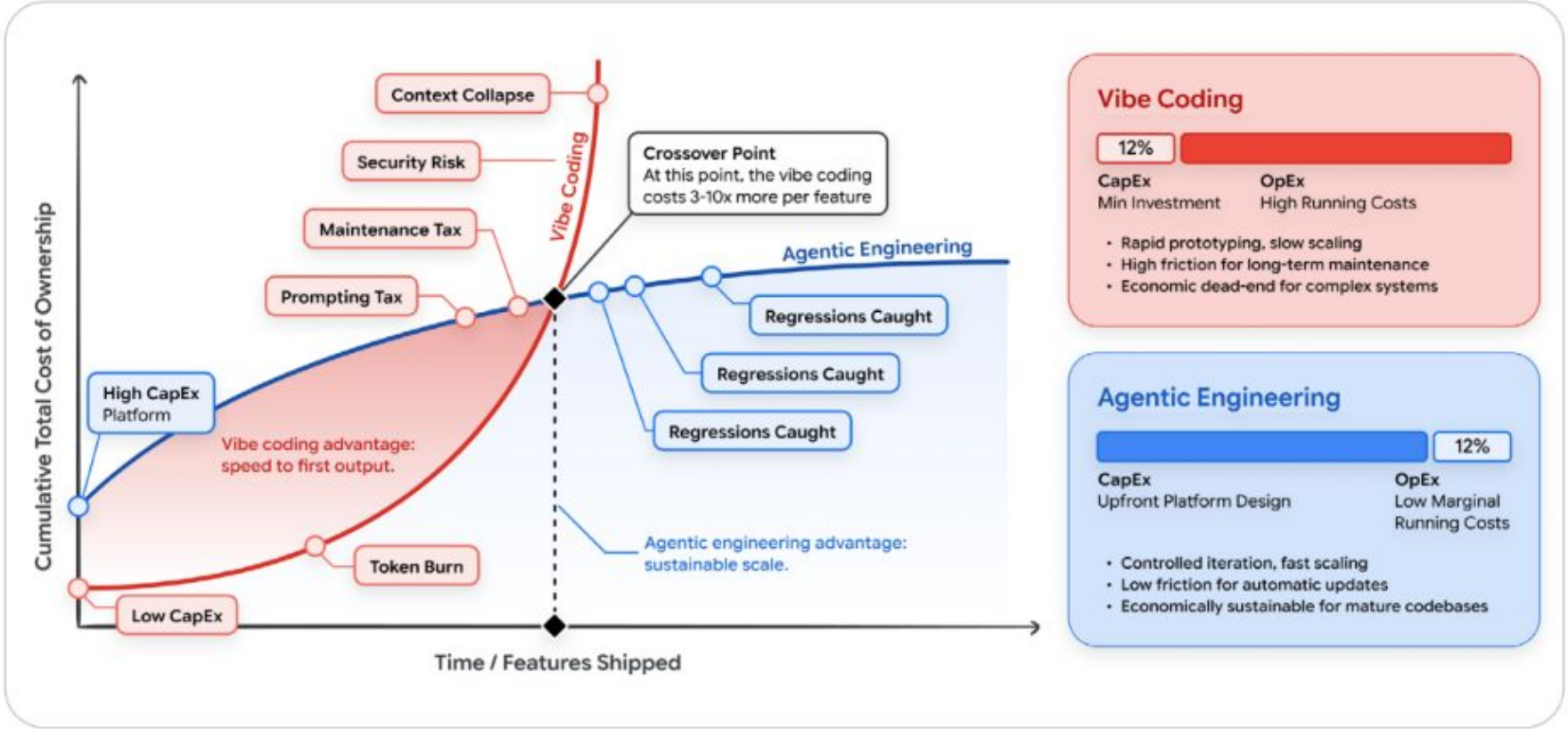


<https://www.kaggle.com/learn-guide/5-day-agents-vibecoding>

Agentic Engineering: Redefinición del Rol del Desarrollador/a



Agentic Engineering: Redefinición de la inversión en AI



<https://www.kaggle.com/learn-guide/5-day-agents-vibecoding>

Día 2: Temas que vimos...

- Ciclo de Vida de Creación de Agentes
- Usos de los modelos de lenguaje en diversos sectores
- IA Conversacional y sus categorías
- Definición de objetivos y alcance del agente
- Requerimientos para la creación de agentes
- Diseño conversacional y experiencia (UX)
- Definición de flujos, interacciones y arquitectura del agente (Root Agent, Flow, Steering, Intent)
- Conversaciones de principio a fin
- Agentes orientados a la ejecución de procesos
- Elección del modelo de lenguaje (Propietarios, Open Models, Open-Source)
- Ventana de contexto y limitaciones
- Model Cards y documentación de modelos
- Alucinaciones, sesgos y técnicas de control (Temperatura, Top-p, Top-k)
- Arquitectura de la solución y microservicios
- MCP (Model Context Protocol): Arquitectura y herramientas
- Comparativa: Herramientas, agentes, subagentes y MCP
- Patrones de agentes y niveles de maduración (Niveles 1-6)
- Patrones de robustez (Supervisor Watchdog, Human in the Loop, Adaptive Retry)
- Registro de herramientas y agentes
- Agentic Engineering y la fábrica de software
- Redefinición del ciclo de vida, rol del desarrollador e inversión en IA